TRNGs para generación de secuencias muy largas

por

Ciro Fabián Bermúdez Márquez

Tesis presentada en cumplimiento parcial de los requisitos para el grado de:

MSc. en Electrónica

en

Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE)

Julio, 2021

Santa María de Tonantzintla, Puebla

Asesores:

Dr. Esteban Tlelo Cuautle

Departamento de Electrónica INAOE

Dr. Cuauhtemoc Mancillas López

Departamento de Electrónica INAOE

©INAOE 2021

Todos los derechos reservados

El autor otorga a INAOE el permiso para reproducir y distribuir este documento.

Agradecimientos

Índice general

Agradecimientos I

Índice general IV

Índice de figuras V

Índice de tablas VII

Lista de códigos IX

Resumen XI

1. Introducción 1

1.1. Fichas bibliográficas 1

1.1.1. Lista 1

1.1.2. Notas 2

1.2. Palabras clave 15

1.3. Clasificación 16

1.4. Justificación 16

1.5. Objetivos 16

1.5.1. Objetivo general 16

1.5.2. Objetivos específicos 16

2. Fundamentos teóricos 17

3. Implementación de intregradores fracionarios 19

3.1. ¿Qué es una FPAA? 19

4. Oscilador caótico utilizando integradores de orden fraccionario 21

4.1. Oscilador caótico basado en funciones no lineales saturadas (SNLF) . . 21

5. Conclusiones 23

A. Códigos 25

Bibliografía **27**

Índice de figuras

Índice de tablas

Lista de códigos

Resumen

Este trabajo de tesis presenta

Capítulo 1

Introducción

1.1. Fichas bibliográficas

1.1.1. Lista

1. 2021 - Luis Gerardo De La Fraga - Designing an authenticated Hash function with a 2D chaotic map. [1]

2. 2017 - Luis Gerardo De La Fraga - Hardware implementation of pseudo-random number generators based on chaotic maps. [2]

3. 2021 - Sundarapandian Vaidyanathan - A 5-D Multi-Stable Hyperchaotic Two-Disk Dynamo System With No Equilibrium Point: Circuit Design, FPGA Realization and Applications to TRNGs and Image Encryption. [3]

4. 2020 - N. Nalla Anandakumar - FPGA-Based True Random Number Generation Using Programmable Delays in Oscillator-Rings. [4]

5. 1999 - Benjamin Jun and Paul Kocher - The Intel Random Number Generator. [5]

6. 2021 - Xinyu Wang - High-Throughput Portable True Random Number Generator Based on Jitter-Latch Structure. [6]

7. 2021 - Adriaan Peetermans - Design and Analysis of Configurable Ring Oscillators for True Random Number Generation Based on Coherent Sampling. [7]

8. 2016 - Oto Petura - A Survey of AIS-20/31 Compliant TRNG Cores Suitable for FPGA Devices. [8]

1.1.2. Notas

1. -

- 2D chaotic map to generate pseudo-random number generators.
- NIST suite 800-22-a and TestU01 tests.
- The generated random sequences are used to implement a stream cipher and are applied to encrypt images.
- Use the random sequences as keys for a hash function based on the pseudo-dot product (PDP). This hash can be used as a message authentication code.
- Development board Nucleo F746ZG, it has STM32F746ZGT6 MCU with up to 216 MHz Cortex-M7F core,
- Throughput of 173.35Kbit/s.
- Images have certain characteristics as data redundancy, strong correlation among adjacent pixels, being less sensitive as compared to the text data, etc.
- The security to encrypt an image depends on the random sequences generated with a given chaotic entity.
- Chaotic maps have been used to create PRNGs. The chaotic maps are a source of entropy to the PRNGs.
- Propose to encrypt image data with the xor operation among image bits and the generated random sequence.
- The simplest way of xoring the image bits with the generated binary random sequence is required to encrypt the image. The requirement is that the binary sequence must be random. If data are xoring with a random sequence, the resulted sequence is also random.
- 2D chaotic map proposed by Sprott

$$\begin{aligned} x_{n+1} &= a_1 + a_2 x_n + a_3 x_n^2 + a_4 x_n y_n + a_5 y_n + a_6 y_n^2 \\ y_{n+1} &= a_7 + a_8 x_n + a_9 x_n^2 + a_{10} x_n y_n + a_{11} y_n + a_{12} y_n^2 \end{aligned} \tag{1.1}$$

- NIST suite 800-22-a and TestU01 tests (Rabbit, Alphabit, and BlockAlphabit).
- The measured throughput was of 12.84 Kbits/s at the MCU working frequency of 16 MHz, or it could be 173.35 Kbits/s working at the maximum clock fre- quency of 216 MHz.

- We can define a function called StreamCipher, which receive as input a message of size n bits; this function internally generates $\lceil n/16 \rceil$ random words and apply xor operation between the message bits and the generated random words.
- Image integrity, a (MAC) is generated by applying a hash function over the input message, using a key K and a random sequence provided by a 2D map.
- A common characteristic of a hash function is that it must give a short fixed length output; with very similar messages, their hash must be very different and, more important, should be impossible to generate a message that produces a given hash.
- The pseudo-dot product (PDP) is used as the basis to implement the hash function, and this is also a message authentication code; the PDP is secure because it is a universal hash function.

2. -

- The usefulness of bifurcation diagrams to implement a pseudo-random number generator (PRNG) based on chaotic maps.
- Details on the selection of the best parameter values to obtain high entropy and positive Lyapunov exponent from the bifurcation diagram of four chaotic maps, namely: Bernoulli shift map, tent, zigzag, and Borujeni maps.
- Implementations in software and in hardware (FPGA).
- The applications of maps for the development of PRNGs have the same goal: improving randomness, high maximum Lyapunov exponent and entropy.
- The bifurcation diagram is quite useful to perform a fast exploration of the control parameter values which produce the output of interest for a specific application of the chaotic map.
- NIST test suite 800-22.
- All these maps share the characteristics to have only a single control parameter, and also a single Lyapunov exponent (LE).
- The Lyapunov exponent can be defined as the average rate of exponential divergence or convergence of very near trajectories in the phase space. Any system with at least one positive LE is defined as a chaotic system, and reflecting its magnitude the temporal scale when its dynamics become unpredictable.

- The LE for a chaotic map can be calculated using the formula:

$$\lambda = \frac{1}{n}\sum_{i=0}^{n-1} \ln\left|\frac{dx_{n+1}}{dx_n}\right| \tag{1.2}$$

- Bernoulli shift map, values inside the range $[-a, a]$

$$x_{n+1} = \begin{cases} bx_n - a & \text{si} \quad x_n \geq 0 \\ bx_n + a & \text{si} \quad x_n < 0 \end{cases} \quad \text{or} \quad x_{n+1} = bx_n - a\,\text{sign}(x_n) \tag{1.3}$$

- Tent map

$$x_{n+1} = \begin{cases} ux_n & \text{si} \quad x_n \in \left[0, \frac{1}{u}\right] \\ \frac{u}{u-1}(1 - x_n) & \text{si} \quad x_n \in \left(\frac{1}{u}, 1\right] \end{cases} \tag{1.4}$$

- Zigzag map

$$x_{n+1} = \begin{cases} -m\left(x_n + \frac{2}{|m|}\right) & \text{si} \quad x_n \in \left(-1, -\frac{1}{|m|}\right] \\ -mx_n & \text{si} \quad x_n \in \left(-\frac{1}{|m|}, \frac{1}{|m|}\right] \\ -m\left(x_n - \frac{2}{|m|}\right) & \text{si} \quad x_n \in \left(\frac{1}{|m|}, 1\right] \end{cases} \tag{1.5}$$

- Borujeni map

$$x_{n+1} = \begin{cases} rx_n(1 - x_n) & \text{si} \quad x_n < 0.5 \\ r(x_n - 0.5)(x_n - 1.5) + \frac{r}{4} & \text{si} \quad x_n \geq 0.5 \end{cases} \tag{1.6}$$

- The entropy measurement is a characteristic of a binary sequence.
- A binary sequence can be generated using the following equation

$$b_{n+1} = \begin{cases} 0 & \text{si} \quad x_{n+1} < q \\ 1 & \text{si} \quad x_{n+1} \geq q \end{cases} \tag{1.7}$$

- The produced sequence is said to be "pseudo-random" because exactly the same sequence can be reproduced using the same initial conditions.
- Once we have a binary sequence, it is possible to calculate its entropy. The entropy is a measure about how unpredictable a binary. sequence is. For a good TRNG or PRNG, the generated bits should be impossible to predict given the previous bits. The entropy can be estimated using the formula:

$$H(b) = -\sum_{i=1}^{n} p_i \log_2 p_i \tag{1.8}$$

where b is a sequence built in this case with symbols $\{0, 1\}$, and with pro-

bability distributions $\{p_1, p_2, ..., p_n\}$. Applying the equation to a sequence, its entropy is equal to 1 if it is an unpredictable sequence, for a totally predictable sequence its entropy is equal to 0.

- Its entropy is calculated with two methods: one that uses a long sequence of 10^6 bits and then calculates the histogram, the occurrence of small sequences of two, three or more binary symbols of length 10 is computed. Sequences of length 1–16 of a total of 1 million bits were used to calculate those histograms. The histograms form the estimated probabilities to calculate the above equation. The taken estimated entropy will converge to the true entropy as the number of taken sequences tend to infinity. The second method uses the contexttree weighting (CTW) algorithm. The taken measurement by this algorithm is related with lossless compressionschemes.
- Then, the performance of existing data compression algorithms is often used as a rough estimate of the entropy of a block of data. Some studies concludes that the CTW method is the most effective to measure the entropy, with the most accurate and reliable results with much less bits, but it takes also more computation time.
- The sequences obtained directly from any of the maps have not entropy equal to 1, as an unpredictable sequence has. Then, a post-processing procedure must be implemented on the generated sequences to increase the entropy of the sequences. We use the bit counting redundancy reduction technique used in a paper. A sequence bit is generated by calculating the XOR of five bits. These sequences have entropy greater than 0.999. We can see that the entropy increases to values very near to 1.0, for all the maps, when this technique is applied.
- First we check which is the number of bits necessary to implement the chaotic map with fixed point arithmetic and maintain its characteristics.
- Spartan 3E (xc3s500e-4fg320).
- Our design could generate pseudo-random bits at the speed of 7.38 Mbps without the RS-232 interface, but with it its speed is limited to 115,200b/s.
- How does the initial value for any map affect the generated binary sequences. For all initial values the entropy is greater than 0.999, and we can observe that x_0 value does not affect the sequence properties.

3. -

- A new 5-D hyperchaotic dynamo system by adding two feedback controllers to the Rikitake 2-disk dynamo system.

- We devise a dual core high speed hybrid true random number generator (TRNG) using Ring and Heun algorithm based on the new 5-D hyperchaotic oscillator on FPGA.
- Recently, chaotic attractors are classified under two categories, viz. (A) self-excited attractors, and (B) hidden attractors. A chaotic attractor is called self-excited when its basin of attraction overlaps with neighborhood of an equilibrium point of the system. Chaotic attractors, which are not self-excited, are called hidden attractors.
- Chaotic and hyperchaotic oscillators have been active topics for research during the last years due to its usefulness in the development of chaotic secure communication systems and other applications that have been implemented using either analog or digital electronics.
- In recent years, great efforts have been carried out on the development of chaos-based true random number generator (TRNG) structures due to the features including noise-like properties and hiding the information signal of chaotic oscillators. Random number generators have been used in many areas including cryptography, applications using the Monte-Carlo method, computer simulations and modeling and numerical analysis application.
- Images are the most common data presentation model. The data represented by images can be protected by using one of the image data hiding techniques or using one of the image encryption models.
- Our new dynamical system is described as follows:

$$\begin{cases} \dot{y_1} &= -ay_1 + y_2y_3 - dy_4 - qy_5 \\ \dot{y_2} &= -ay_2 + (y_3 - b)y_1 - dy_4 - qy_5 \\ \dot{y_3} &= 1 - y_1y_2 \\ \dot{y_4} &= cy_1 \\ \dot{y_5} &= py_1 \end{cases} \tag{1.9}$$

- FPGA implementation of the new 5-D hyper-chaotic dynamo system given by applying three different methods, namely: forward-Euler, trapezoidal and fourth-order Runge-Kutta. Each numerical method provides different accuracy and requires a good estimation of the step- size to diminish numerical errors.
- Using the FPGA Cyclone IV EP4CGX150DF31C7.
- The designed Heun based discrete mathematical model of a new 5-D hyperchaotic dynamo system. Heun algorithm is a two-step algorithm. Dual

entropy core high speed TRNG unit designed using the 5-D new hyper chaotic oscillator and ring oscillator structures using Heun algorithm on FPGA

- FPGA XC7VX485T-2-FFG1761 chip of the Virtex-7 VC707 family.
- Can generate 416 Mbit random numbers in 1 second on the Virtex-7 VC707 Evaluation Platform.
- NIST 800-22-a.
- Correlation coefficient of neighbouring pixels C_{pc} is a measure used to assessment the meaningful of an image which normal images have C_{pc} value near to 1 in every direction while in cipher images with a well-designed encryption scheme should near to 0. To calculate the outcomes of C_{pc} in every direction of the cipher and plain images, we picked randomly 10^4 pairs of neighbouring pixels in per direction.

$$C_{pc} = \frac{\sum_{i=1}^{N}(p_i - \overline{p})(c_i - \overline{c})}{\sqrt{\sum_{i=1}^{N}(p_i - \overline{p})^2 \sum_{i=1}^{N}(c_i - \overline{c})^2}} \tag{1.10}$$

  where N indicates to the entire number of adjacent pixel pairs in any direction and c_i, p_i are points to the values of neighbouring pixels.
- To estimate the sensitivity of tiny changes in the plain image on its corresponding ciphered one, two tools are applied: "Number of Pixels Change Rate" (NPCR) and "Unified Average Changing Intensity" (UACI).
- Histogram Test.
- Information Entropy Analysis.
- Occlusion Attakcs.
- Key Sensitivity Analyses.

4. -

- New and efficient method to generate true random numbers on field programmable gate array (FPGA) by utilizing the random jitter of free-running oscillators as a source of randomness.
- The free-running oscillator rings incorporate programmable delay lines (PDLs) to generate large variation of the oscillations and to introduce jitter in the generated ring oscillators clocks. The main advantage of the proposed TRNG utilizing PDLs is to reduce correlation between several equal length oscillator rings, and thus improve the randomness qualities.
- Von Neumann corrector as post-processor is employed to remove any bias in the output bit sequence.

- Xilinx Spartan-3A FPGAs (XC3S400A-4FTG256)
- 6 Mb/s throughput with 0.999 per bit entropy rate.
- Typical TRNGs use a single source of entropy and post-processing operation. Commonly used sources of entropy are thermal noise, metastability, clock jitter in circuits, and chaos.
- Randomness is first extracted from the physical noise source and then this is interpreted into raw random bit-stream using a sampler (digitization). In practice, the raw random bit-stream usually does not bear good quality of randomness. Therefore, auxiliary post-processing operations, such as von Neumann corrector or hash function is required to improve the quality of the output TRNG bit stream and increase its randomness.
- A number of field programmable gate array (FPGAs) prototypes of TRNG with post-processing designs have been proposed. These designs derive entropy from the jitter of Ring Oscillators (RO) or the metastability of flip-flops which is caused by setup or hold time violations of flip-flops (FFs).
- There are different problems that might arise in the construction of a TRNG based on oscillator rings implemented in FPGAs.
- For example, the entropy of the output bit sequence from the TRNG would be drastically reduced when equal length oscillator rings configured in FPGAs are highly correlated with each other due to identical delays. To address this issue, we use programmable delay lines (PDLs) in the oscillator rings in the current work. This creates higher variation in RO oscillations from cycle to cycle and causes jitter in the generated RO clocks. Also, the output of the RO's are not correlated with each other due to the incorporation of the PDLs in the oscillator rings for each sampling clock which produces higher randomness.
- The LUT based primitives are instantiated in hardware description language (HDL) as described in Spartan-3 Libraries Guide for HDL Designs.
- Programmable Delay Lines (PDLs). The internal variations of FPGA Look-Up Tables (LUT's) can be generated from changes in the LUT's propagation delays under different input.
- Jitters can be used to generate a stream of truly random bits using D flip-flop (DFF) based sampler for sampling the output of a high frequency oscillator. The quality of generated true random bits can be improved by employing multiple free-running RO's.
- This design passes the NIST statistical tests without post-processing and employs reduced number of RO's. However, it has similar mathematical com-

plexity to the original design and hence similar associated problems such as mutual dependence of rings and correlation between the rings, which cause a lack of entropy at the output of the TRNG. The randomness qualities of the original TRNG can also be improved at the cost of higher hardware resources or a lower throughput.

- It is imperative to note that the RO based TRNGs, although exciting, are extremely limited in terms of randomness when identical RO's are employed. Equal length oscillator rings configured in FPGA are highly correlated with each other due to identical delays and therefore the XOR of the output from these rings returns mostly zeros. This leads to poor random- ness from the design. We show in this brief that the a TRNG incorporating PDL's can overcome this problem.
- Throughput of 6 Mbps at 24 MHz Operating Frequency.
- Correlations Test.
- Measures of the Quality of Randomness, NIST statistical test suite.

5. -

- Good cryptography requires good random numbers. Almost all cryptographic protocols require the generation and use of secret values that must be unknown to attackers. Random number generators are required to generate public/private keypairs for asymmetric. Keys for symmetric and hybrid cryptosystems are also generated randomly. RNGs are also used to create challenges, nonces (salts), padding bytes, and blinding values.
- Because security protocols rely on the unpredictability of the keys they use, random number generators for cryptographic applications must meet stringent requirements. The most important is that attackers, including those who know the RNG design, must not be able to make any useful predictions about the RNG outputs. In particular, the apparent entropy of the RNG output should be as close as possible to the bit length.
- Most random number sources actually utilize a pseudorandom generator (PRNG). PRNGs use deterministic processes to generate a series of outputs from an initial seed state. Because the output is purely a function of the seed data, the actual entropy of the output can never exceed the entropy of the seed. It can, however, be computationally infeasible to distinguish a good PRNG from a perfect RNG.
- A true random number generator (TRNG) uses a non-deterministic source to produce randomness. Most operate by measuring unpredictable natural

processes, such as thermal (resistance or shot) noise, atmospheric noise, or nuclear decay. The entropy, trustworthiness, and performance all depend on the TRNG design.

- A PRNG by itself will be insecure without a TRNG for seeding. Seeding requires a source of true randomness, since it is impossible to create true randomness from within a deterministic system.
- In many cases, system designers are faced with a trade-off between security and convenience. For example, to avoid having to collect fresh seed data each time the program loads, many software applications store their seed material on the hard drive where there can be a risk of compromise. The best sources for unpredictable data involve timing user processes, which add undesirable complexity to user interfaces.
- The Intel RNG primarily samples thermal noise by amplifying the voltage measured across undriven resistors. In addition to a large random component, these measurements are correlated to local pseudorandom environmental characteristics, such as electromagentic radiation, temperature, and power supply fluctuations.
- The Intel RNG uses a random source that is derived from two free-running oscillators, one fast and one much slower. The thermal noise source is used to modulate the frequency of the slower clock. The variable, noise-modulated slower clock is used to trigger measurements of the fast clock. Drift between the two clocks thus provides the source of random binary digits.
- The initial random measurements are processed by a hardware corrector based on a concept proposed by John von Neumann to produce a balanced distribution of "0" and "1" bits. A von Neumann corrector converts pairs of bits into output bits by converting the bit pair $[0, 1]$ into an output 1, converting $[1, 0]$ into an output 0, and outputting nothing for $[0, 0]$ or $[1, 1]$. The corrector is a simple way to generate statistically balanced outputs from data with residual bias.

6. -

- Although TRNGs based on ring oscillators (ROs) and phase-locked loops (PLLs) have the advantages of small resource overhead and high throughput, there are problems such as instability of randomness and poor portability. To improve the randomness, portability, and throughput of a random number generator, we design a TRNG whose randomness is generated by the oscillation of self-timed rings (STRs) and accurately extracted by a jitter-latch structure

- The circuit structure used to generate random phenomena in TRNGs is called an entropy source. Commonly used entropy sources in FPGAs are Phase locked loops (PLLs), digital clock management (DCM), ring oscillators (ROs), and self-timed rings (STRs).
- A TRNG using PLL as the entropy source circuit, it shows good randomness, but its portability is limited for the PLL design, and it must be readjusted often to ensure a highly random output. Similar problems are observed in entropy source circuits based on ROs. Strict path delay is required in TRNGs. A TRNG based on a programmable delay line (PDL) was proposed by Anandakumar and Rozic proposed one based on a fast carry chain, with the intent to reduce the delay caused by process deviation. These require complicated placing and routing steps to increase the randomness. The portability of these structures was limited, which led to the dependence of randomness on the degree of delay adjustment.
- Considering that STRs are more robust than ROs, many scholars have conducted research on them as entropy sources of TRNGs.
- Cherkaoui used an STR instead of an RO to design a true random number generator, but the design consumed many resources and much power. Martin used coherent sampling STRs to extract true random numbers, which reduced the overhead of TRNG while ensuring randomness, but its throughput rate was low. Zhang used fast forward on FPGAs to capture the signal edge positions of STR loops as logic primitives. However, this method required manual placement and lost the portability of the structure. Liu proposed a staged-running TRNG structure which used the unstable state in STRs. The piecewise running strategy adopted in the design affected the throughput of the generator, and the randomness quality was not ideal. However, they proposed the idea of quantifying jitter as unstable data bits in the process of state transition. Inspired by the design of Liu, this paper proposes a TRNG based on a jitter-latch structure to improve the portability and the randomness difference between devices.
- Throughput of 100 Mbps.
- NIST Test.

7. -

- These TRNGs are usually based either on the unpredictability of metastable memory elementsor on the timing jitter present in free-running oscillators.
- NIST 800-22 test.

- From an implementation aspect, TRNG designers need to consider the feasibility of the entropy source on an FPGA, portability across different FPGA families and vendors, design constraints, and design effort. Unlike most digital designs, TRNGs usually require some manual setup or placement and routing constraints. For example, designs based on Self-Timed Ring oscillators (STRs) and delay chains require placement constraints that have to be set up for each FPGA family, thus limiting the portability of these designs. In addition, some entropy sources do not work correctly on all locations on an FPGA.
- COherent Sampling ring Oscillator based TRNG (COSO-TRNG)
- The existing COSO-TRNG implementations can achieve a throughput of the order of 1 Mbit/s, requiring only minimal chip area. The entropy source consists of a structure that generates two oscillating signals with similar periods. In practice, a generic way of creating these two signals is by using two identically designed Ring Oscillators (ROs).
- In this article, we propose a set of highly portable, easy-to-use architectures of a COSO entropy source without requiring any device-specific blocks such as PLLs, carry-chains, or DSPs, without any placement and routing constraints and without the need for a manual search procedure.

8. -

- RNGs are typically composed of an analog physical source of randomness, a digitizer, and an optional entropy conditioning block. The source of randomness, digitization, and the entropy harvesting mechanism depend to a large extent on the selected technology, a standard or even a recommended TRNG does not exist. Depending on the characteristics of the source of randomness and the quality of the digital noise (the output of the digitizer), designers select the entropy conditioning method that will enhance the statistical properties of generated numbers.
- In the past, during the design and the security evaluation and certification process, the principle of the TRNG and its implementation were only evaluated statistically: the generated numbers were tested using standard test suites such as FIPS 140-1, NIST SP 800-22, DIEHARD, and DIEHAR-DER.
- However, this approach is not suitable for modern data security systems for several reasons: 1) post-processing can mask considerable weaknesses in the source of randomness; 2) generic statistical tests can only evaluate the statistical quality of the numbers that are generated and not their entropy; 3) high-end standard statistical tests are complex and hence both expensive

and slow, plus they require huge data sets. Consequently, they are only executed occasionally or on demand and only on selected sets of data of limited size.

- Our aim was to select such generators and to fairly evaluate the difficulties related to their implementation in different FPGA technologies, their area, output bit rate, power requirements, and the statistical quality of their output.
- To compare TRNG principles and their implementations in different FPGA families as fairly as possible, the evaluation boards should have the same topology and should use as few components introducing deterministic noise as possible (e.g. should be powered using low noise power supplies). This rigorous approach was not applied in recent papers, in which five different retail evaluation boards, which contain switching power supplies, were used. This means the claimed performance of TRNGs evaluated in the above mentioned papers was specific to particular boards and operating conditions.
- Our contribution: 1) We implement AIS-20/31 compliant TRNG cores in different FPGA families; 2) we compare the generators using always the same low noise dedicated hardware; 3) we propose two new evaluation criteria for better comparison of TRNGs – the energy efficiency and the product of the entropy and bit rate; 4) to ensure reproducibility of our results, we have made the VHDL code of all generators adapted to individual families freely available (open source).
- Many TRNG cores have been published up to now, but many are not suitable for implementation in digital devices like FPGAs. This is the case of generators using analog components like analog amplifiers or analog to digital converters used in chaos-based TRNGs. Furthermore, some digital designs require full custom digital technology and are thus not feasible in FPGAs. On the other hand, some generators use the specific features of specific FPGA family and are not directly feasible in other FPGA families. Our objective was to select sufficiently generic principles that are feasible in all recent FPGA families.
- Such TRNG cores must fulfill several requirements: 1) their design must be simple and comprehensible, the source of randomness must be clearly defined; 2) the underlying random process must be stationary and the stochastic model must be feasible; 3) the raw binary signal must be available for further off-line and on-line testing. In addition, it is helpful if the source of randomness (e.g. the clock jitter coming from the thermal noise) is quantifiable – its measurement inside the device can provide a basis for fast and efficient

embedded statistical tests.

- We pre-selected TRNG cores that use oscillating circuitries: single-event ring oscillators (i.e. standard ring oscillators), multi-event ring oscillators with signal collisions (i.e. transition effect ring oscillators), multi-event ring oscillators without signal collisions (i.e. self-timed rings), and phase-locked loops (PLLs).
- Classification:
  a) Ring Oscillator Based Elementary TRNG (ERO-TRNG)
  b) Ring Oscillator Coherent Sampling Based TRNG (COSO-TRNG)
  c) Multi-Ring Oscillator Based TRNG (MURO-TRNG)
  d) Coherent Sampling Based TRNG Using PLLs (PLL-TRNG)
  e) Transition Effect Ring Oscilaltor Based TRNG (TERO-TRNG)
  f) Self Timed Ring Based TRNG (STR)

9. -
   - -
   - -
   - -
   - -
   - -
   - -
10. -
    - -
    - -
    - -
    - -
    - -
    - -
11. -
    - -
    - -
    - -

- -
- -
- -

12. -

- -
- -
- -
- -
- -
- -

1.2. Palabras clave

1. (TRNG) True Random Number Generator.
2. (PRNG) Pseudo Random Number Generators.
3. MAC) Message Authentication Code.
4. Authentication Tag.
5. NIST) The National Institute of Standards and Technology.
6. Entropy
7. (CTW) Contexttree weighting algorithm.

Los True Random Number Generators (TRNGs) se utilizan en aplicaciones criptográficas para crear llaves aleatorias (random keys), parámetros en protocolos de desafió-respuesta, valores de relleno y máscaras.

Para construir aplicaciones de este tipo usualmente se utilizan Field-Programmable Gate Arrays (FPGAs) debido a su flexibilidad, alto rendimiento y bajo costo en comparación con otras alternativas. Sin embargo la implementación de TRNGs en FPGAs es un reto debido a los limitados recursos y técnicas específicas de TRNG disponibles para el diseñador. Debido a la disponibilidad de recursos digitales en las FPGAs, estos TRNGs se basan normalmente en la imprevisibilidad de los elementos de memoria metaestable o en el jitter de tiempo presente en los osciladores de funcionamiento libre (free-running oscillators).

La salida de un TRNG debe ser capaz de pasar pruebas estadísticas, sin embargo, no basta con pasar estas pruebas estadísticas para garantizar la seguridad de un TRNG, la seguridad debe justificarse mediante un modelo estocástico de la fuente de entropía.

1.3. Clasificación

1. Implementación en FPGA

   a) Based on Self-Timed Ring oscillators (STRs) and delay chains.

   b) COherent Sampling ring Oscillator based TRNG (COSO-TRNG).

   c) Based Phase-Locked Loop– (PLL)

En todos los artículos que se han leído se tiene la misma visión que cuando se trata con TRNGs es importante tener una **alta entropía** de los números generados, **alto rendimiento** (throughput), **portabilidad**, y pasar las pruebas estándar.

En muchas mayoría de los casos es necesario realizar un posprocesamiento para aumentar o disminuir alguna de las características del TRNG.

No existe una señal digital ideal en un circuito real. Debido al ruido térmico, los flancos de subida y bajada de la forma de onda de muestreo a veces se adelantan o se retrasan, lo que se llamado jitter.

1.4. Justificación

1.5. Objetivos

1.5.1. Objetivo general

Objetivos

1.5.2. Objetivos específicos

- Objetivo

Capítulo 2

Fundamentos teóricos

Esto es un ejemplo de una cita [1]

Capítulo 3

Implementación de intregradores fracionarios

3.1. ¿Qué es una FPAA?

Capítulo 4

Oscilador caótico utilizando integradores de orden fraccionario

4.1. Oscilador caótico basado en funciones no lineales saturadas (SNLF)

Capítulo 5

Conclusiones

Apéndice A

Códigos

Bibliografía

[1] L. G. D. la Fraga, C. Mancillas-López, and E. Tlelo-Cuautle, "Designing an authenticated hash function with a 2d chaotic map," *Nonlinear Dynamics*, vol. 104, pp. 4569–4580, may 2021.

[2] L. G. de la Fraga, E. Torres-Pérez, E. Tlelo-Cuautle, and C. Mancillas-López, "Hardware implementation of pseudo-random number generators based on chaotic maps," *Nonlinear Dynamics*, vol. 90, pp. 1661–1670, aug 2017.

[3] S. Vaidyanathan, A. Sambas, B. Abd-El-Atty, A. A. A. El-Latif, E. Tlelo-Cuautle, O. Guillen-Fernandez, M. Mamat, M. A. Mohamed, M. Alcin, M. Tuna, I. Pehlivan, I. Koyuncu, and M. A. H. Ibrahim, "A 5-d multi-stable hyperchaotic two-disk dynamo system with no equilibrium point: Circuit design, FPGA realization and applications to TRNGs and image encryption," *IEEE Access*, vol. 9, pp. 81352–81369, 2021.

[4] N. N. Anandakumar, S. K. Sanadhya, and M. S. Hashmi, "FPGA-based true random number generation using programmable delays in oscillator-rings," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, pp. 570–574, mar 2020.

[5] B. Jun and P. Kocher, "The intel random number generator," *Cryptography Res. Inc., San Francisco, CA, USA*, 1999.

[6] X. Wang, H. Liang, Y. Wang, L. Yao, Y. Guo, M. Yi, Z. Huang, H. Qi, and Y. Lu, "High-throughput portable true random number generator based on jitter-latch structure," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, pp. 741–750, feb 2021.

[7] A. Peetermans, V. Rožić, and I. Verbauwhede, "Design and analysis of configurable ring oscillators for true random number generation based on coherent sampling," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 14, pp. 1–20, jun 2021.

[8] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *2016 26th Inter-*

national Conference on Field Programmable Logic and Applications (FPL), IEEE, aug 2016.